

# CSC633 MACHINE LEARNING



TEXAS A&M UNIVERSITY

Department of Computer  
Science & Engineering

# CSCE 633: Machine Learning

Deep Learning Models

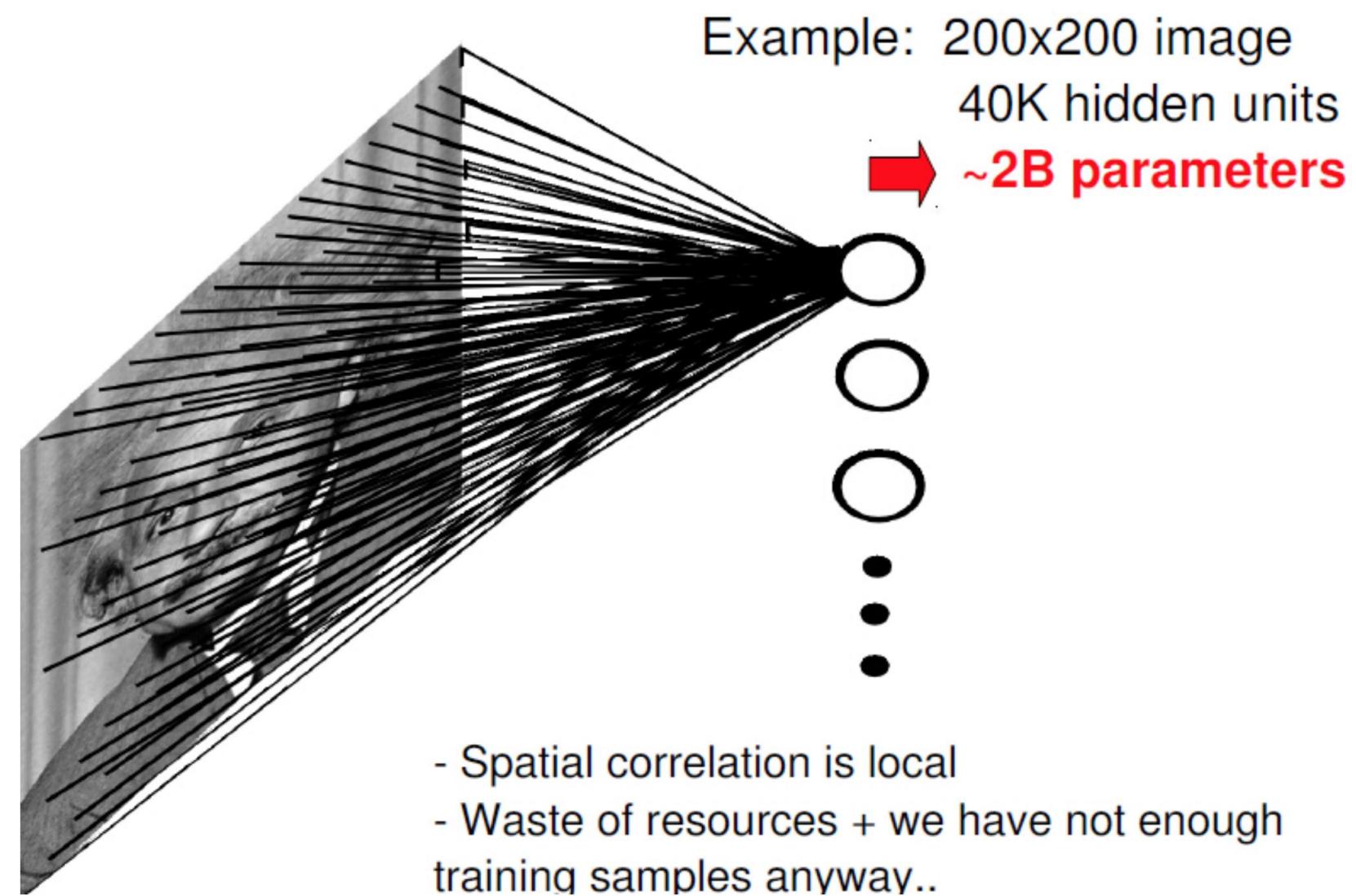
Texas A&M University

# Outline

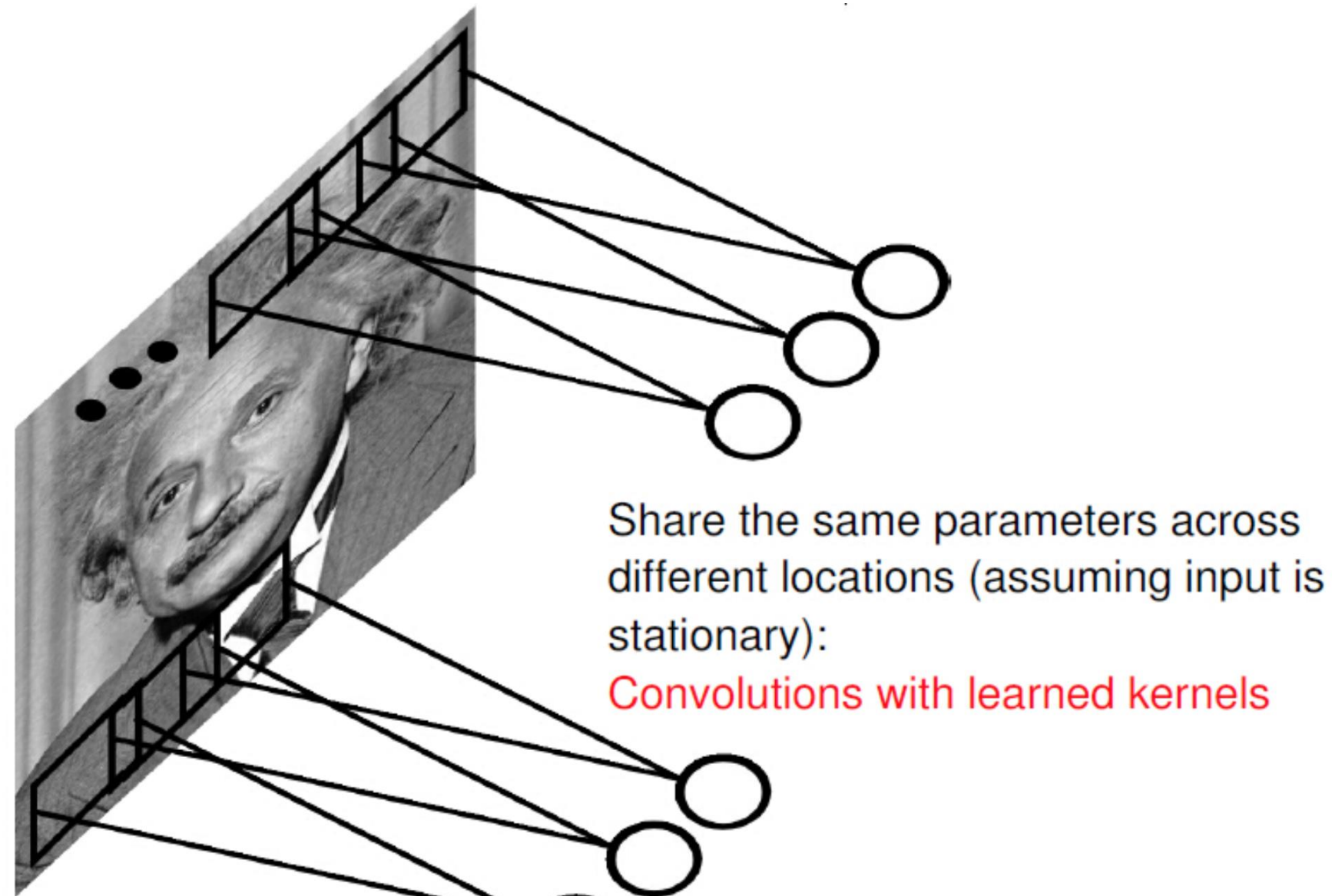
---

- History of Deep Learning
- Convolutional Neural Networks
- Recurrent Neural Networks
- Transfer Learning
- Multi-task Learning

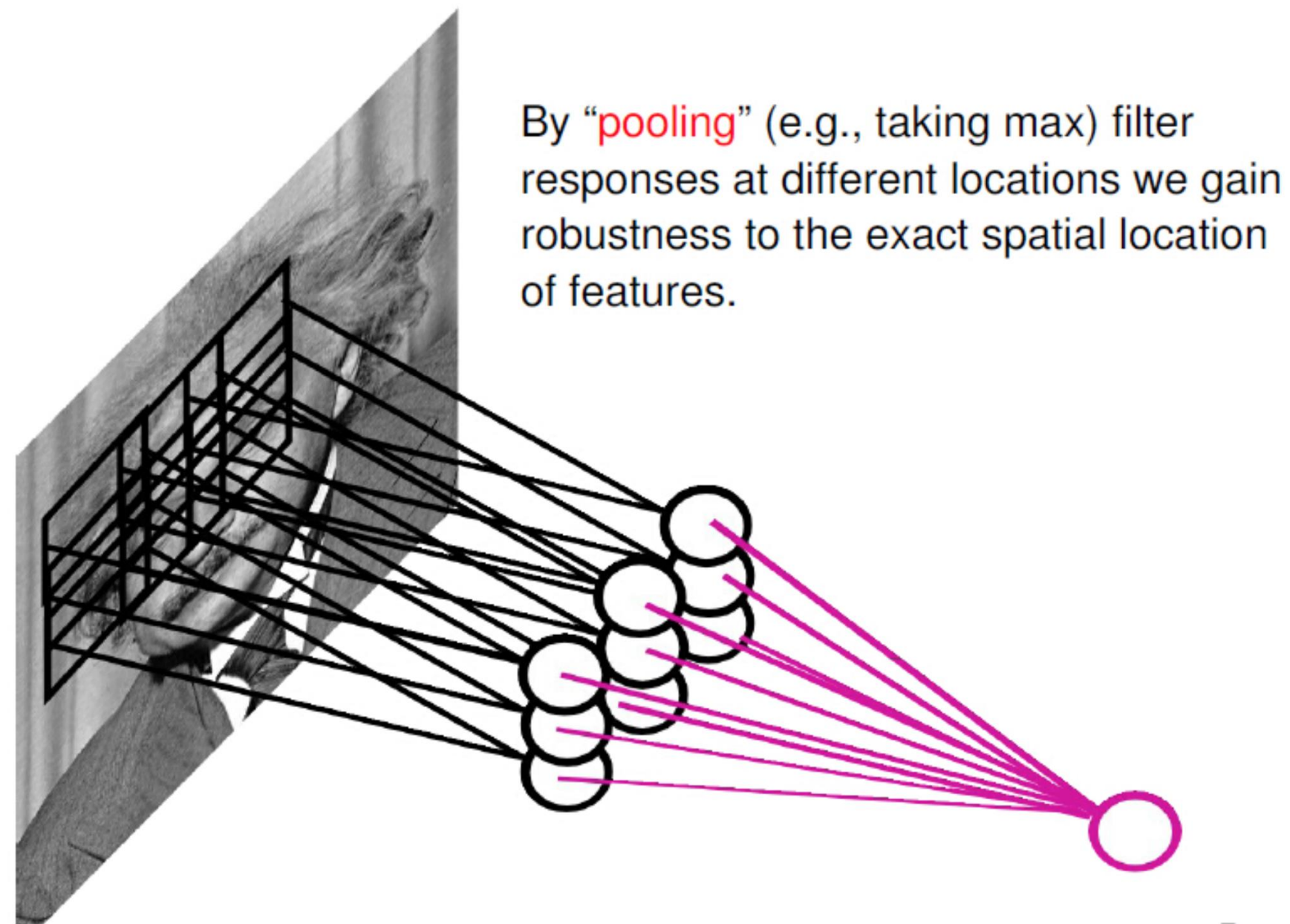
# Fully Connected Layer



# Convolutional Layer

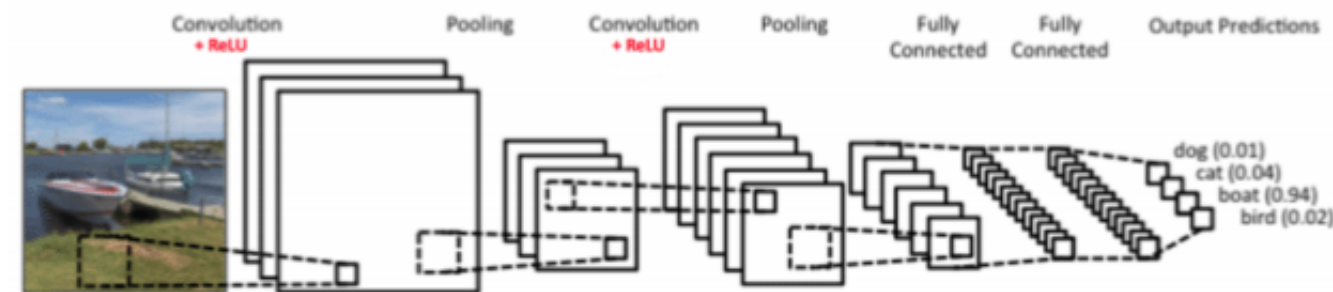


# Pooling Layer



# Convolutional Neural Networks (1)

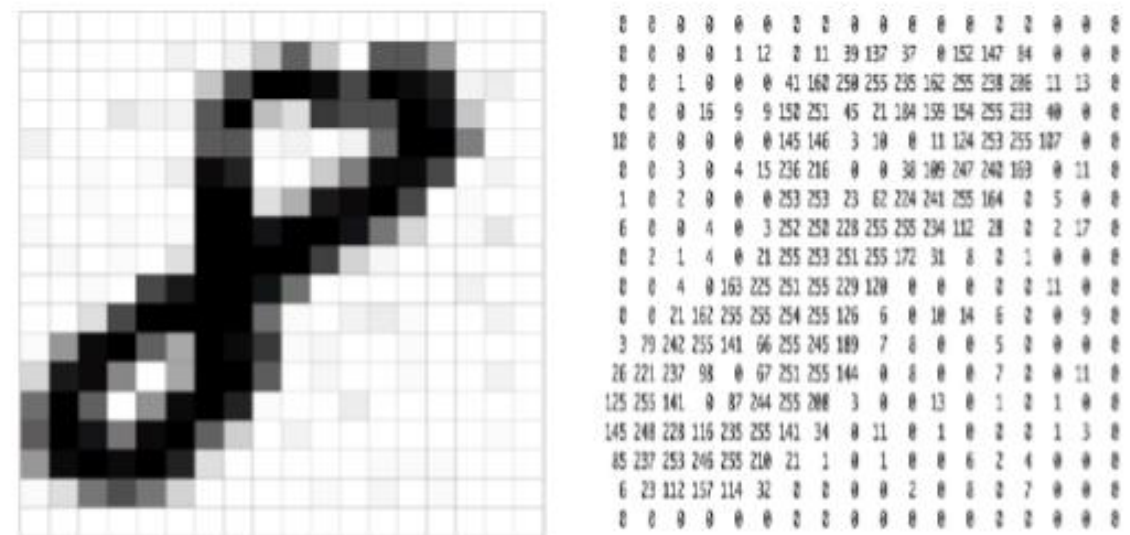
- Similar to regular neural networks
  - made up of neurons, each with an input and an activation function
  - have weights and biases to be learned
  - have a loss function on the last (fully-connected) layer
- Explicit assumption that the inputs are images
  - explicit assumption that the inputs are **images**
  - vastly reduce the amount of parameters in the network



# Convolutional Neural Networks (2)

## Image representation

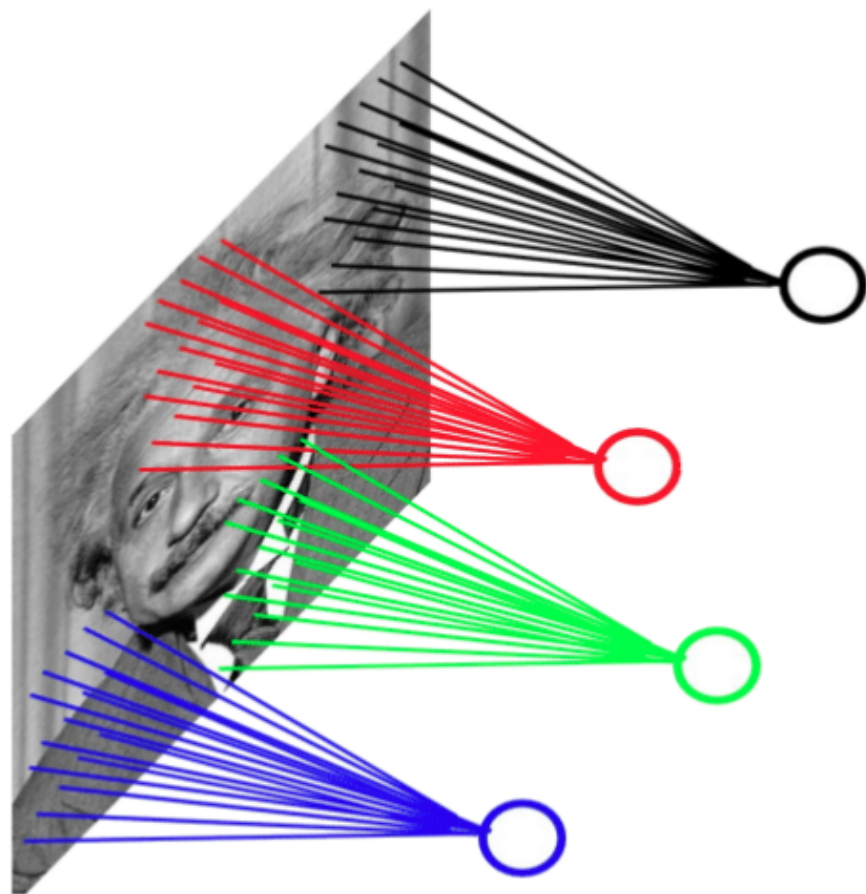
- Grayscale image (1-channel)
  - 2d-matrix
  - each pixel ranges from 0 to 255 - 0: black, 255: white
- Color image (3-channel, RGB)
  - three 2d-matrices stacked over each other
  - each with pixel values ranging between 0 and 255



# Convolutional Neural Networks (3)

## Idea 1: Convolution

A **locally** connected neural network with image input

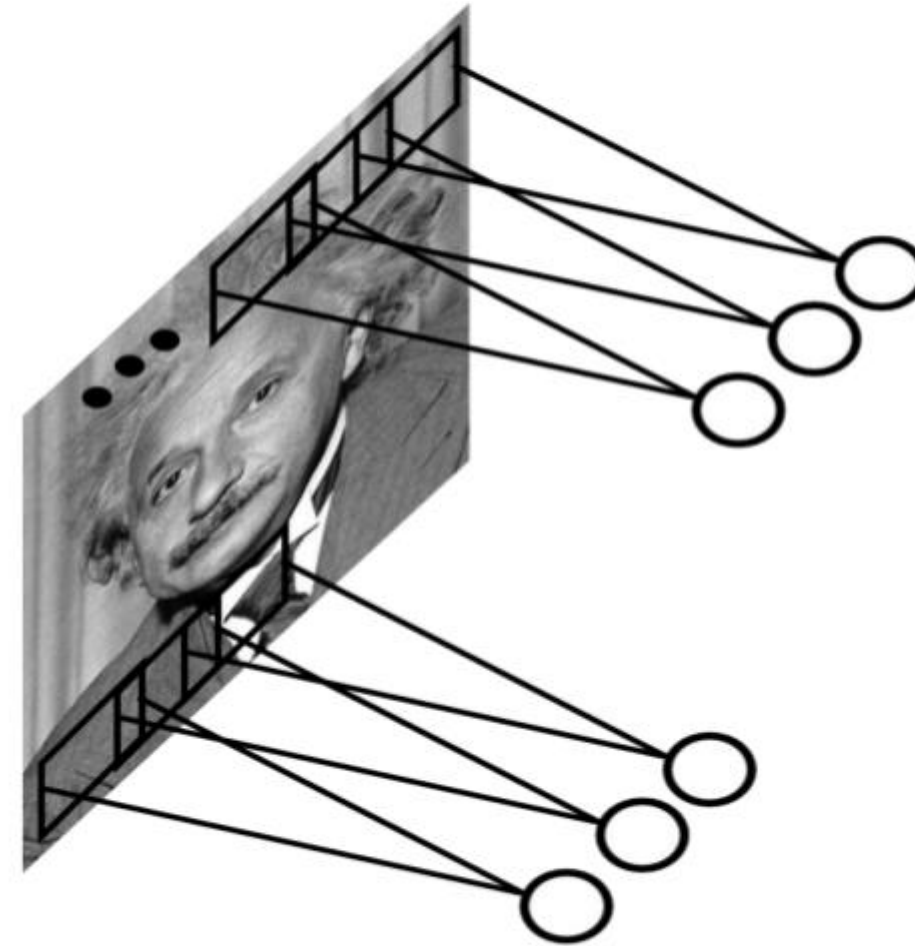
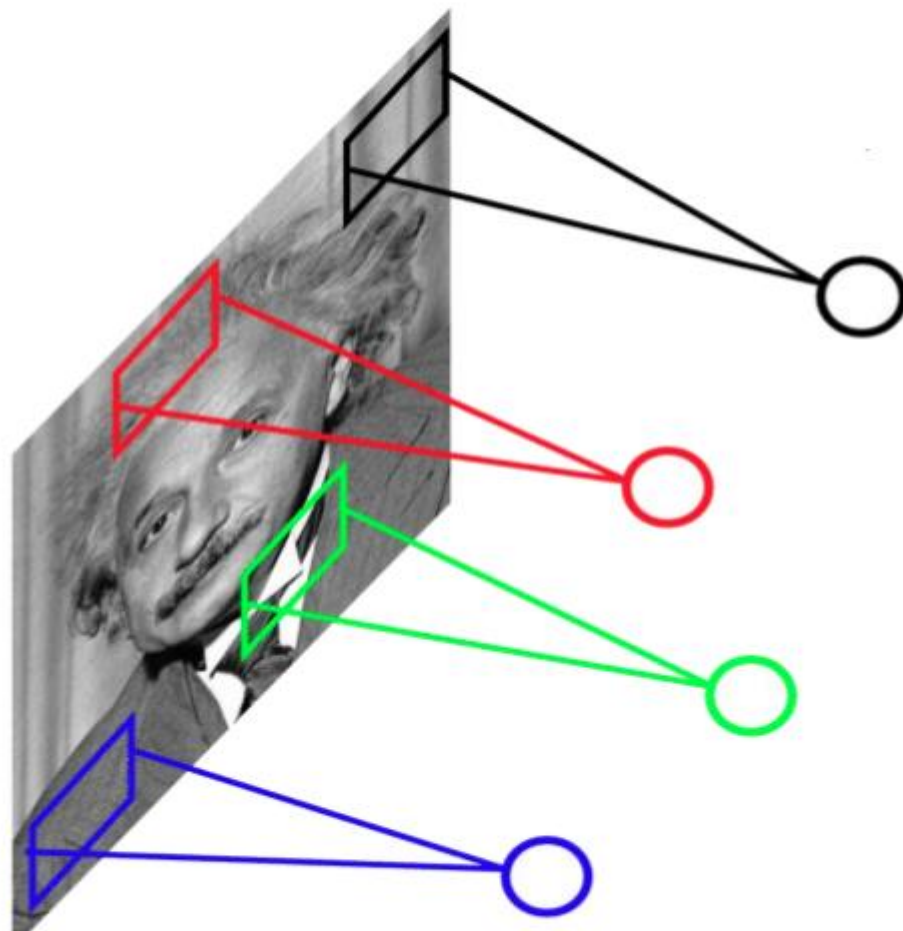


- $1000 \times 1000$  image, 1M hidden units,  $10 \times 10$  filter size  $\rightarrow 10^8$  parameters
- Since spatial correlation is local, we can significantly simplify this

# Convolutional Neural Networks (4)

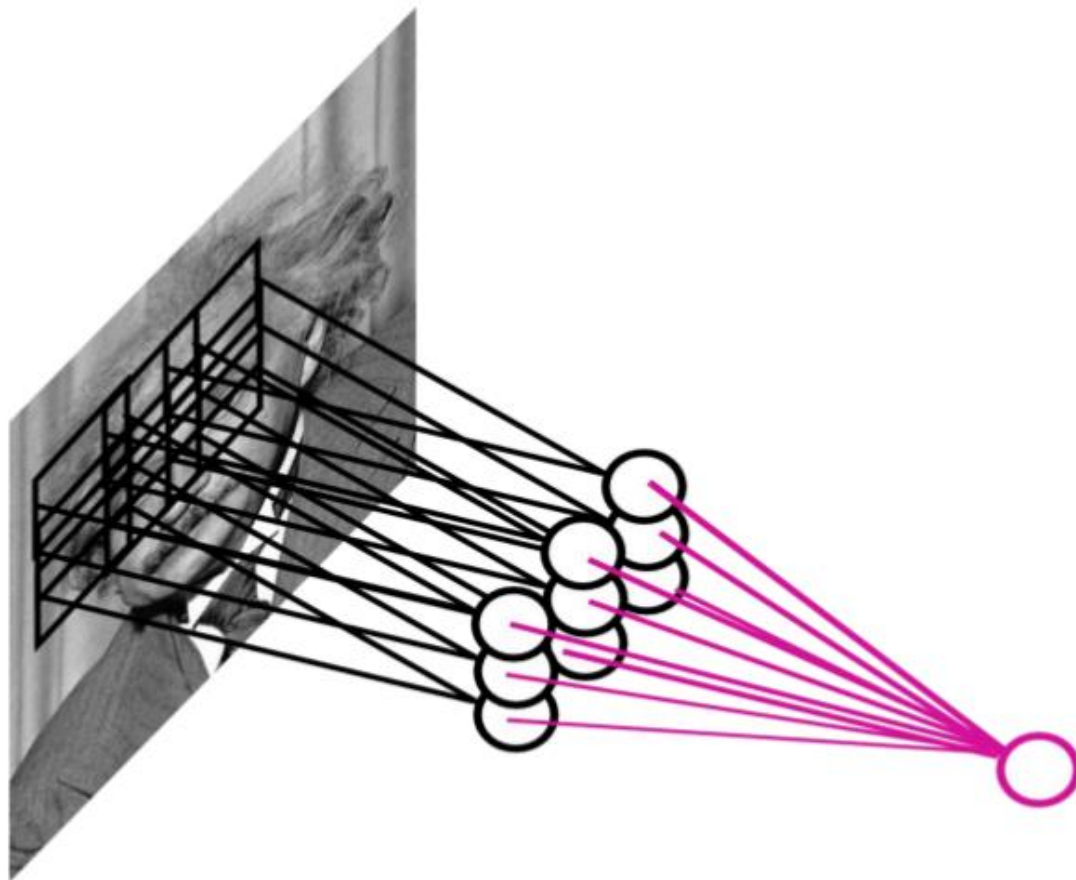
## Idea 2: Weight sharing

- **Stationarity:** Statistics are similar at different locations
- Share the same parameters across different locations



# Convolutional Neural Networks (5)

## Idea 3: Max-pooling



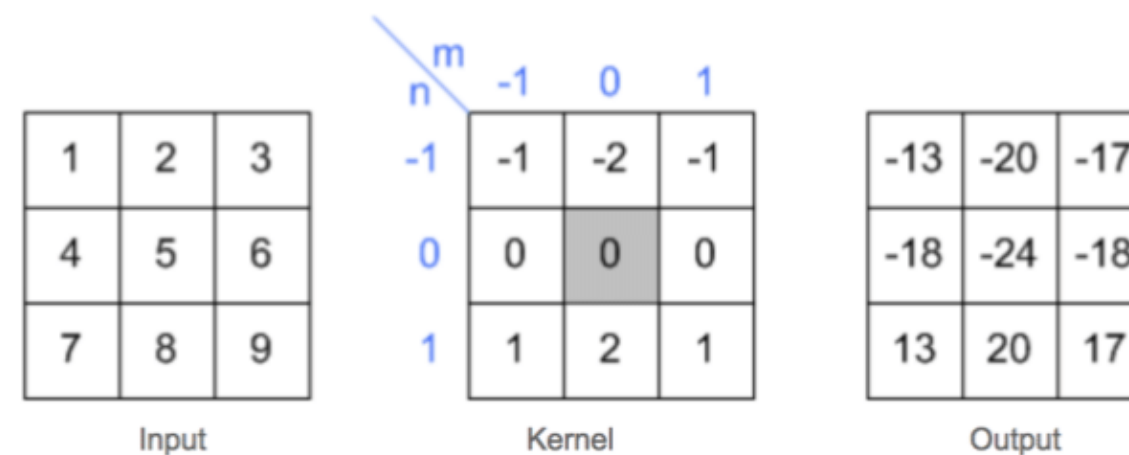
- Let us assume filter is an “eye” detector
- How can we make the detection robust to the exact location of the eye?
- By **pooling** (e.g., max or average) filter responses at different locations we gain robustness to the exact spatial location of features

# Convolutional Neural Networks (6)

## Convolution 2d-example

- Convolution is the mathematical operation that implements filtering
- Given an input image  $x[m, n]$  and an impulse response  $h[m, n]$  (filter or kernel), the convolution output can be written as

$$y[m, n] = x[m, n] * h[m, n] = \sum_{j=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} x[i, j] h[m - i, n - j]$$



[http://www.songho.ca/dsp/convolution/convolution.html#convolution\\_2d](http://www.songho.ca/dsp/convolution/convolution.html#convolution_2d)

# Convolutional Neural Networks (7)

## Convolution 2d-example

|    |    |    |   |
|----|----|----|---|
| 1  | 2  | 1  |   |
| 0  | 0  | 0  |   |
| -1 | -2 | -1 |   |
|    | 7  | 8  | 9 |

$$\begin{aligned}y[0,0] &= x[-1,-1] \cdot h[1,1] + x[0,-1] \cdot h[0,1] + x[1,-1] \cdot h[-1,1] \\&\quad + x[-1,0] \cdot h[1,0] + x[0,0] \cdot h[0,0] + x[1,0] \cdot h[-1,0] \\&\quad + x[-1,1] \cdot h[1,-1] + x[0,1] \cdot h[0,-1] + x[1,1] \cdot h[-1,-1] \\&= 0 \cdot 1 + 0 \cdot 2 + 0 \cdot 1 + 0 \cdot 0 + 1 \cdot 0 + 2 \cdot 0 + 0 \cdot (-1) + 4 \cdot (-2) + 5 \cdot (-1) = -13\end{aligned}$$

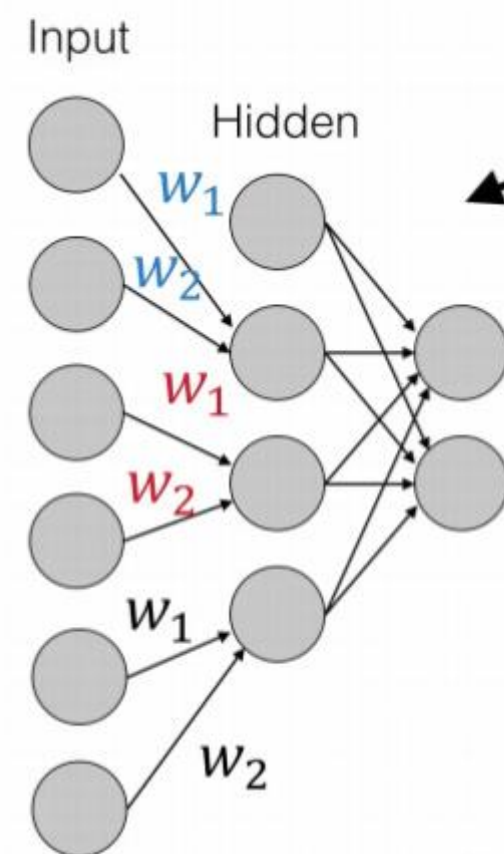
|    |    |    |   |
|----|----|----|---|
| 1  | 2  | 1  |   |
| 0  | 0  | 0  |   |
| -1 | -2 | -1 |   |
|    | 7  | 8  | 9 |

$$\begin{aligned}y[1,0] &= x[0,-1] \cdot h[1,1] + x[1,-1] \cdot h[0,1] + x[2,-1] \cdot h[-1,1] \\&\quad + x[0,0] \cdot h[1,0] + x[1,0] \cdot h[0,0] + x[2,0] \cdot h[-1,0] \\&\quad + x[0,1] \cdot h[1,-1] + x[1,1] \cdot h[0,-1] + x[2,1] \cdot h[-1,-1] \\&= 0 \cdot 1 + 0 \cdot 2 + 0 \cdot 1 + 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 0 + 4 \cdot (-1) + 5 \cdot (-2) + 6 \cdot (-1) = -20\end{aligned}$$

[http://www.songho.ca/dsp/convolution/convolution.html#convolution\\_2d](http://www.songho.ca/dsp/convolution/convolution.html#convolution_2d)

# Convolutional Neural Networks (8)

## Image 1d-convolution

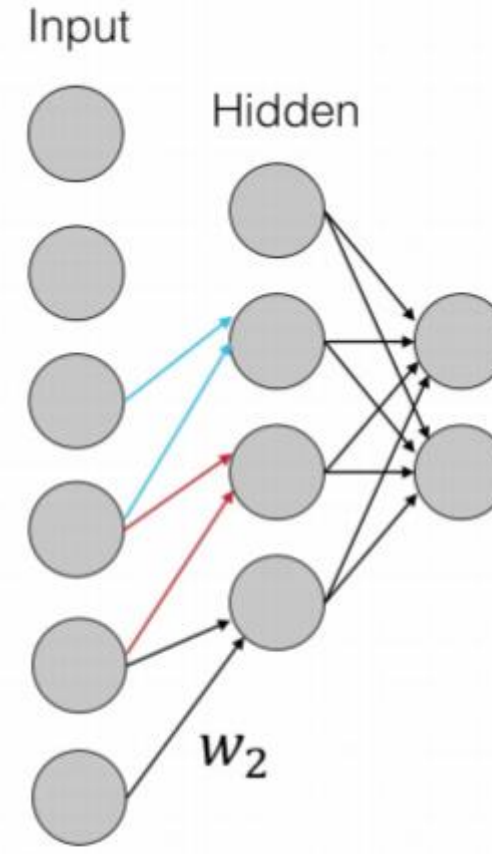


1-d convolution with

- filters: 1
- filter size: 2
- stride: 2

1-d convolution with

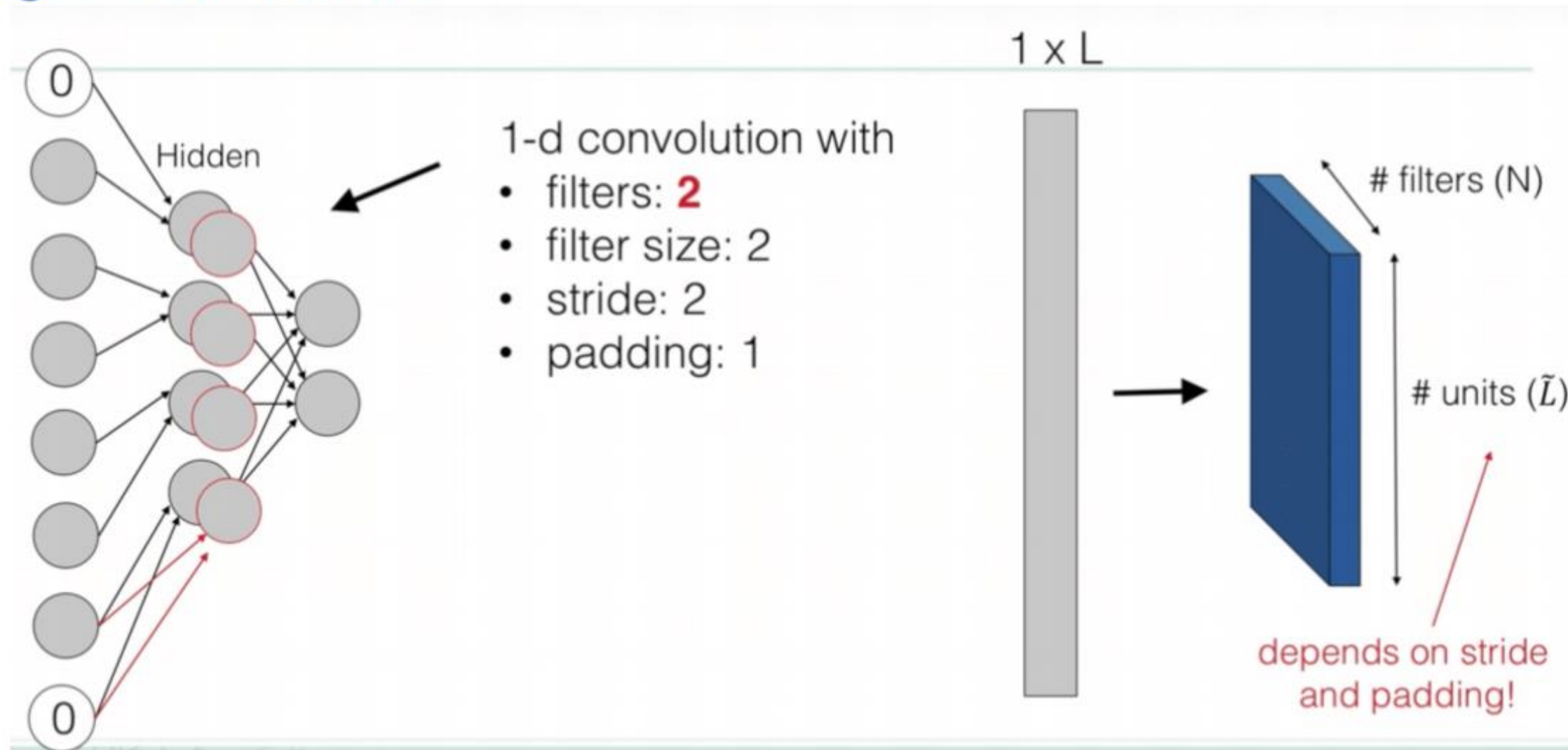
- filters: 1
- filter size: 2
- stride: **1**



<https://www.nervanasys.com/convolutional-neural-networks/>

# Convolutional Neural Networks (9)

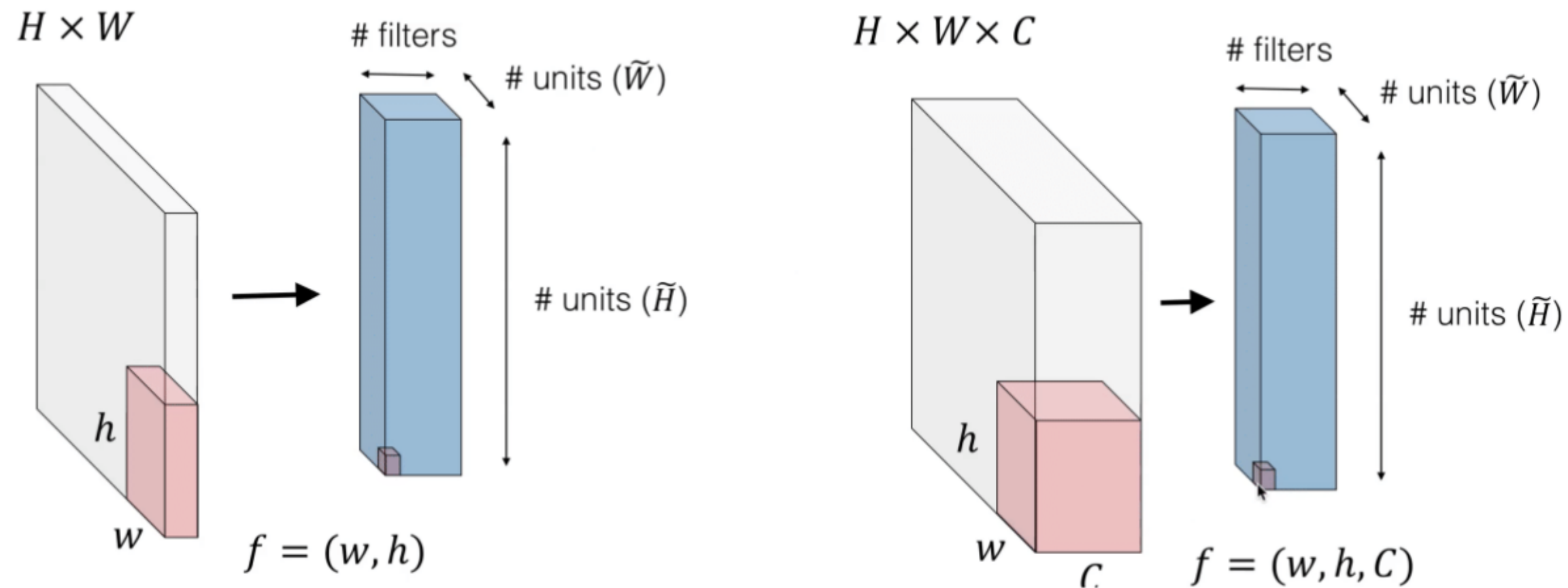
## Image 1d-convolution



<https://www.nervanasys.com/convolutional-neural-networks/>

# Convolutional Neural Networks (10)

## Image 2d-convolution



<https://www.nervanasys.com/convolutional-neural-networks/>

Also check:

<http://cs231n.github.io/assets/conv-demo/index.html>

<https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/> (figure 6)

# Convolutional Neural Networks (11)

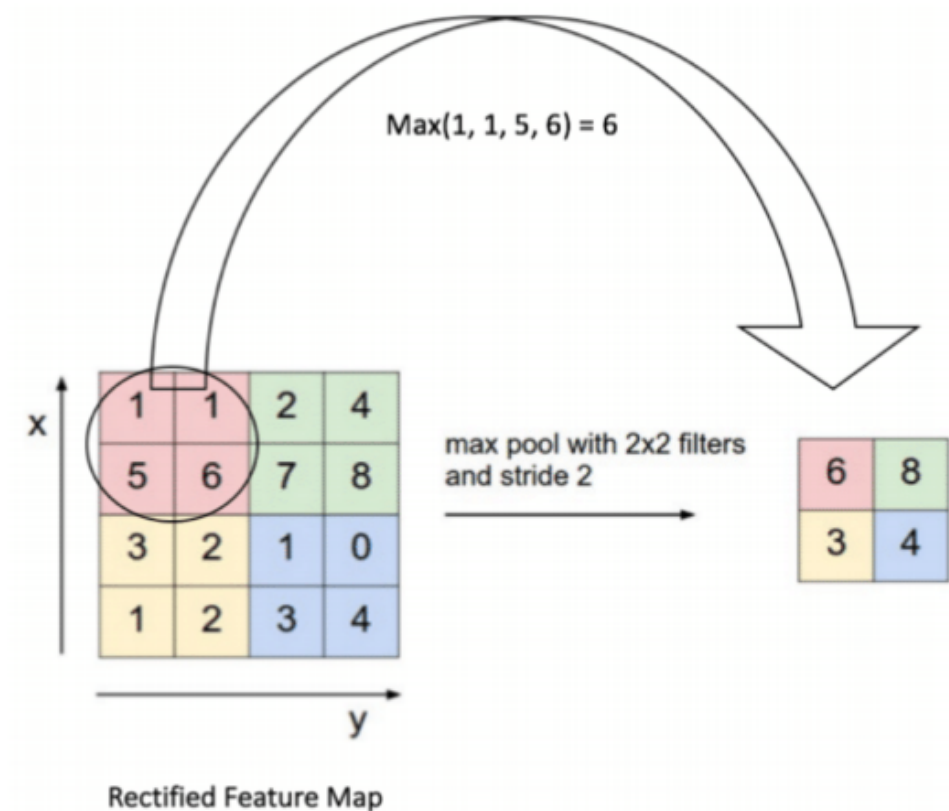
## Image 2d-convolution hyperparameters

- Depth: the number of filters we use for the convolution operation
- Stride: the number of pixels by which we slide our filter matrix over the input
- Zero-padding: padding the input matrix with zeros around the border, so that we can apply the filter to bordering elements of our input image matrix

# Convolutional Neural Networks (12)

## Spatial Pooling (also called subsampling or downsampling)

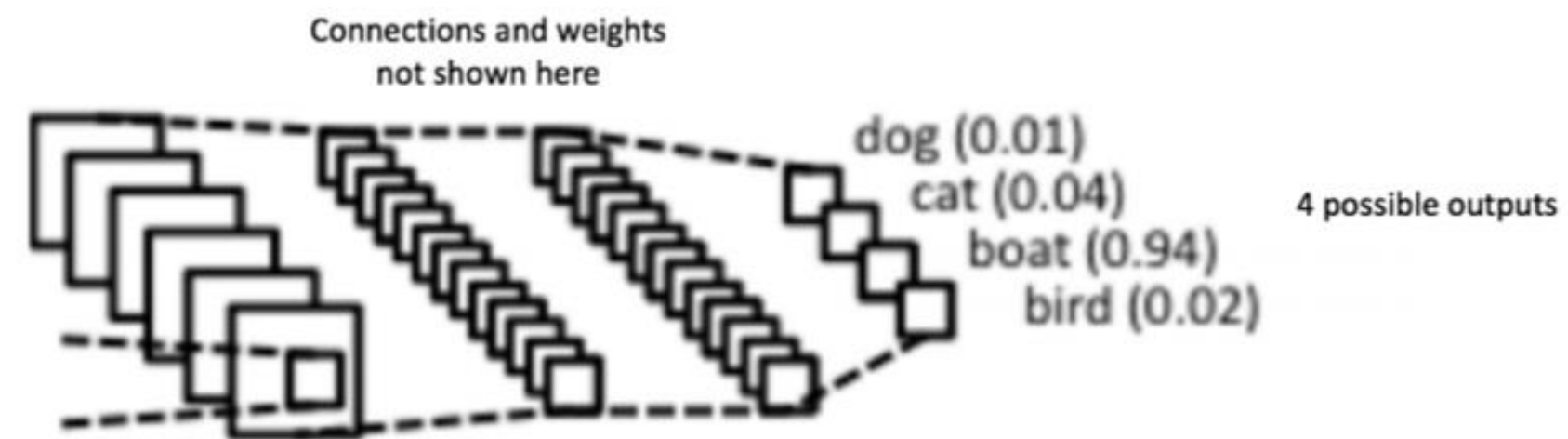
- Reduces the dimensionality of each feature map but retains the most important information
- Can be of different types: Max, Average, Sum etc.
- Makes the input representations (feature dimension) smaller and more manageable
- Promotes an almost scale invariant representation of the image



# Convolutional Neural Networks (13)

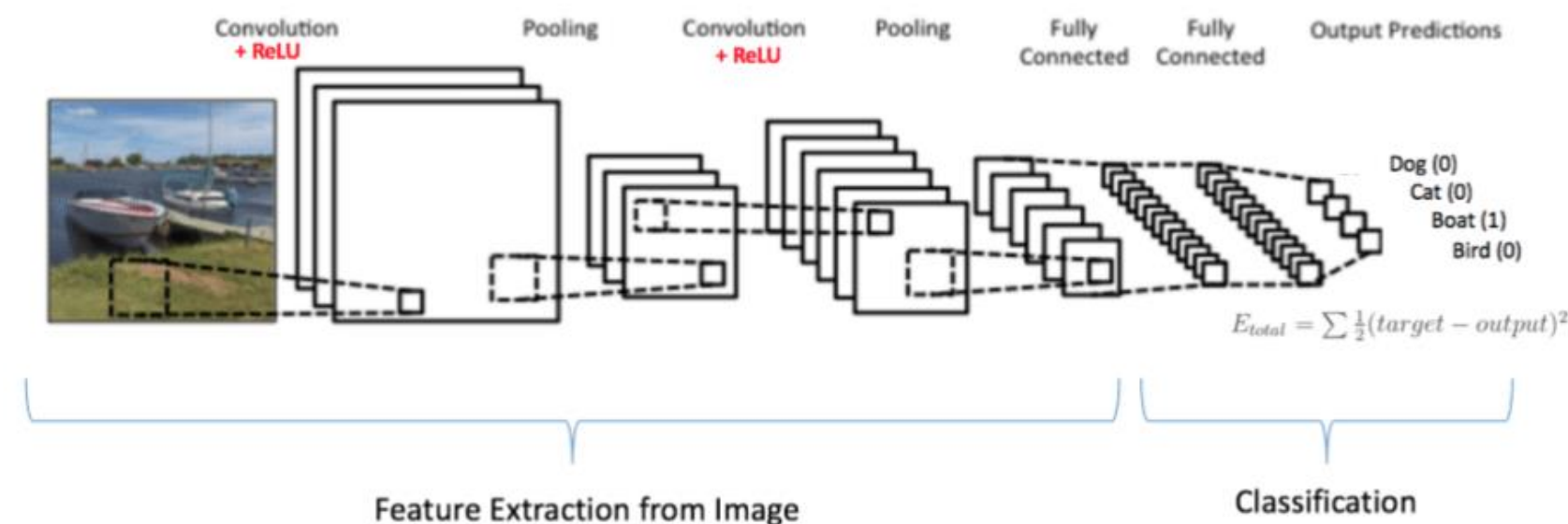
## Final fully connected layer

- Traditional multilayer perceptron
- Yields the classification/regression result

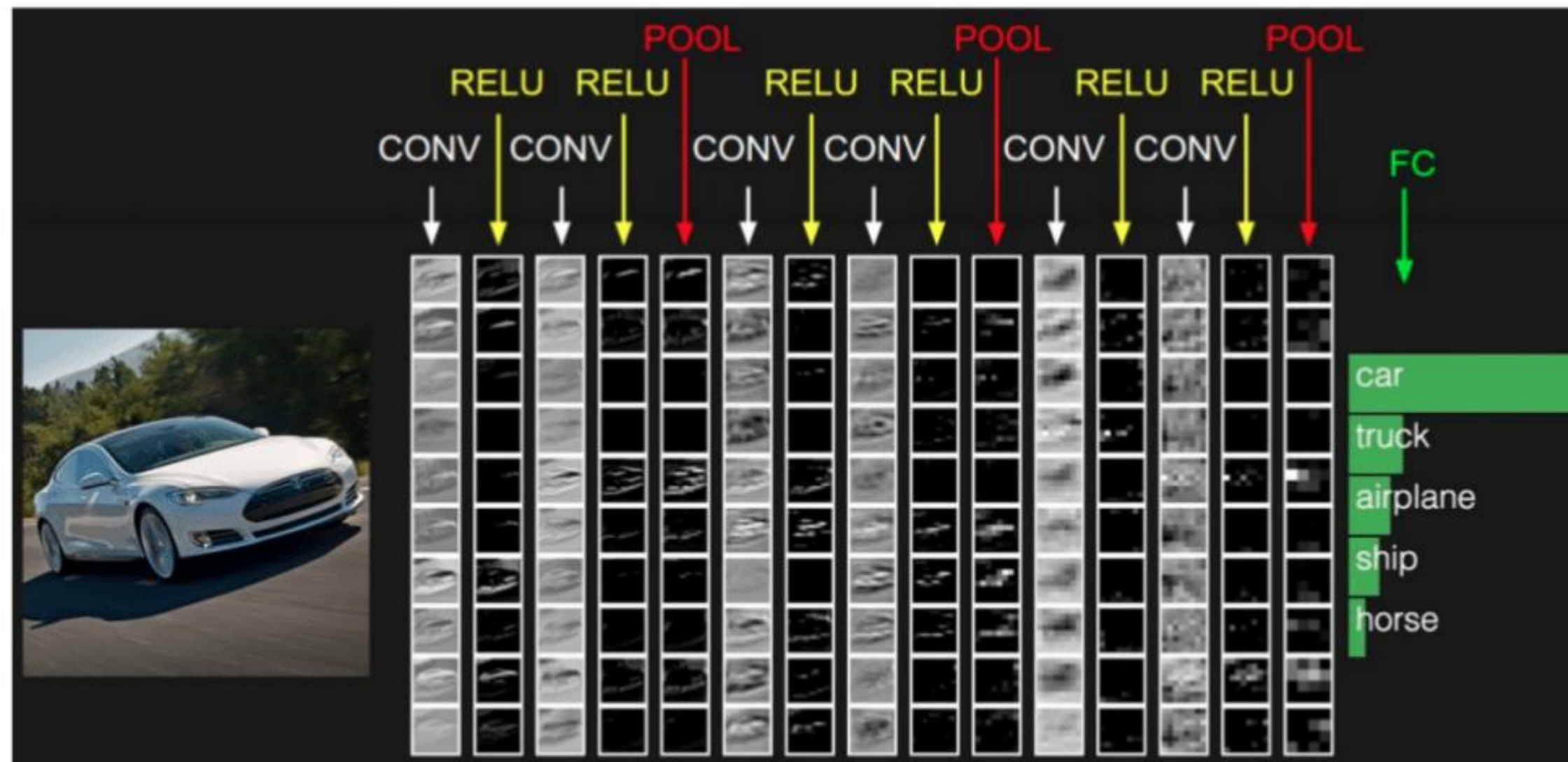


# Convolutional Neural Networks (14)

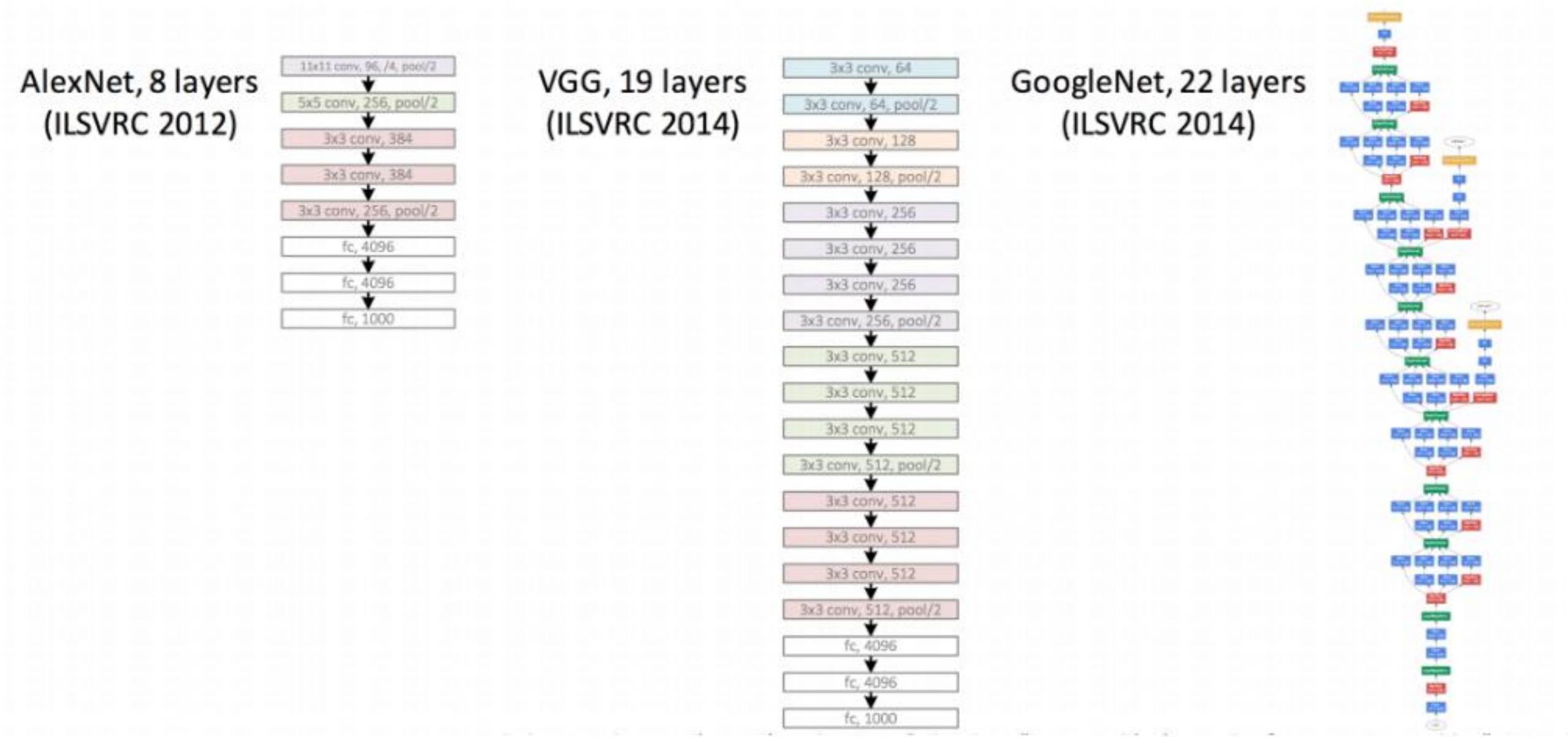
- **Step 1:** Initialize weights
- **Step 2:** Take first image as input and go through the forward propagation step (convolution, ReLU and pooling operations along with forward propagation in the fully connected layer) and finds the output probabilities for each class
- **Step 3:** Calculate the total error at the output layer
- **Step 4:** Use backpropagation to update the weights, which are adjusted in proportion to their contribution to the total error
- **Step 5:** Repeat Steps 1-4 for all train images



# Convolutional Neural Networks (15)

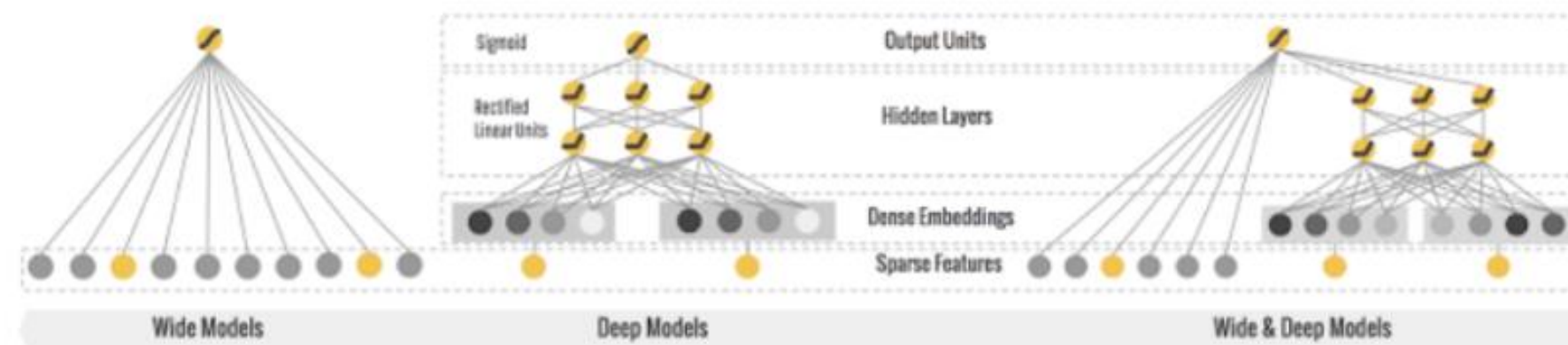


# Convolutional Neural Networks (16)

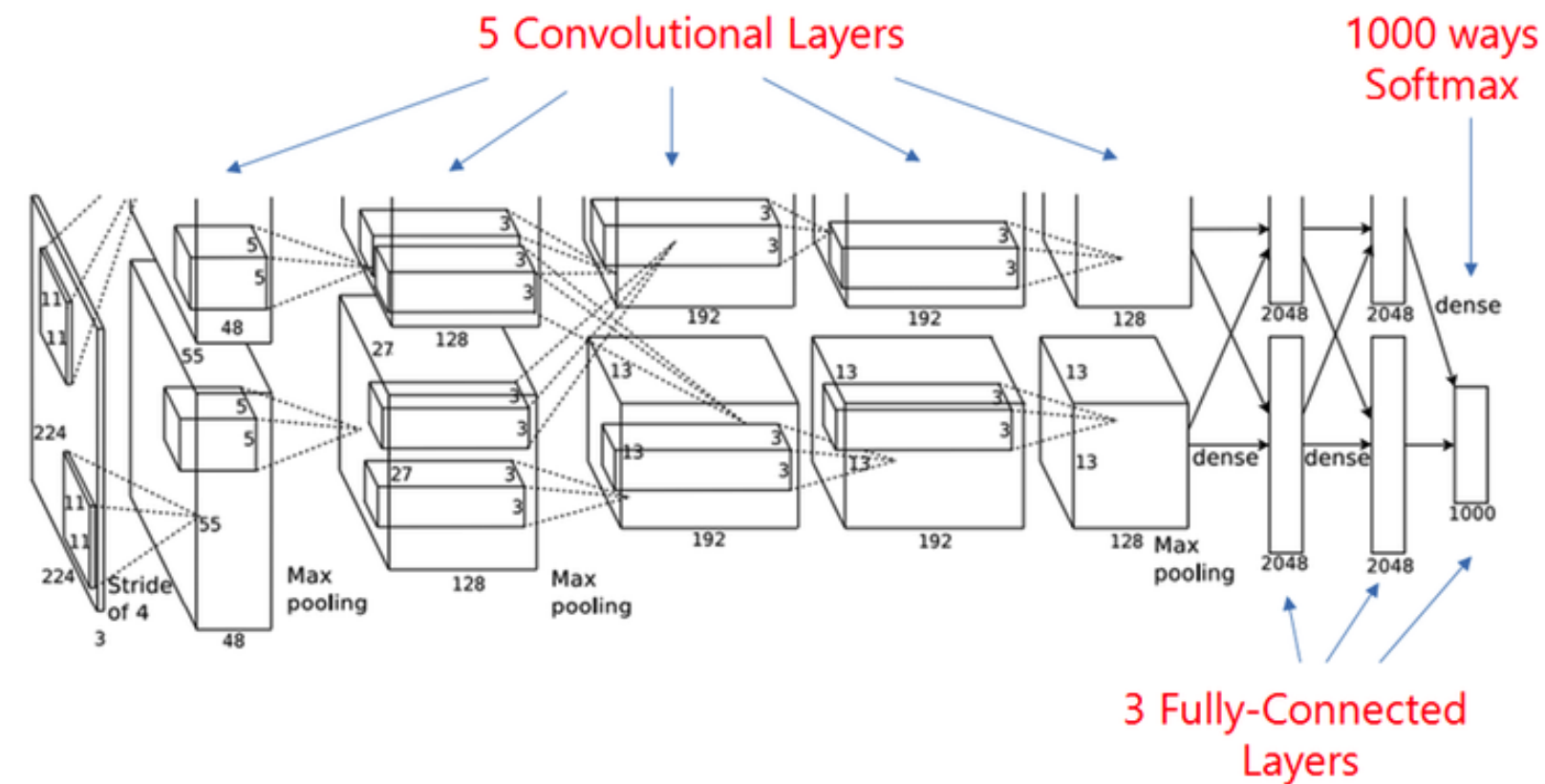


# Wide and deep models

- Google: Wide & Deep Learning (2016)
- generic large-scale regression and classification problems with sparse inputs (categorical features with a large number of possible feature values), such as recommender systems, search, and ranking problems
- The wide model takes into account a wide set of cross-product feature transformations to capture how the co-occurrence of a query-item feature pair correlates with the target label
- The deep feed-forward neural network learns lower-dimensional dense representations for every query and item.



# AlexNet, 2012



- The **FIRST** winner deep model in computer vision, and one of the most classical choices for domain experts to adapt for their applications
- 5 convolutional layers + 3 fully-connected layers + softmax classifier
- **Key Technical Features:** ReLU, dropout, data augmentation

# VGG-Net, 2014

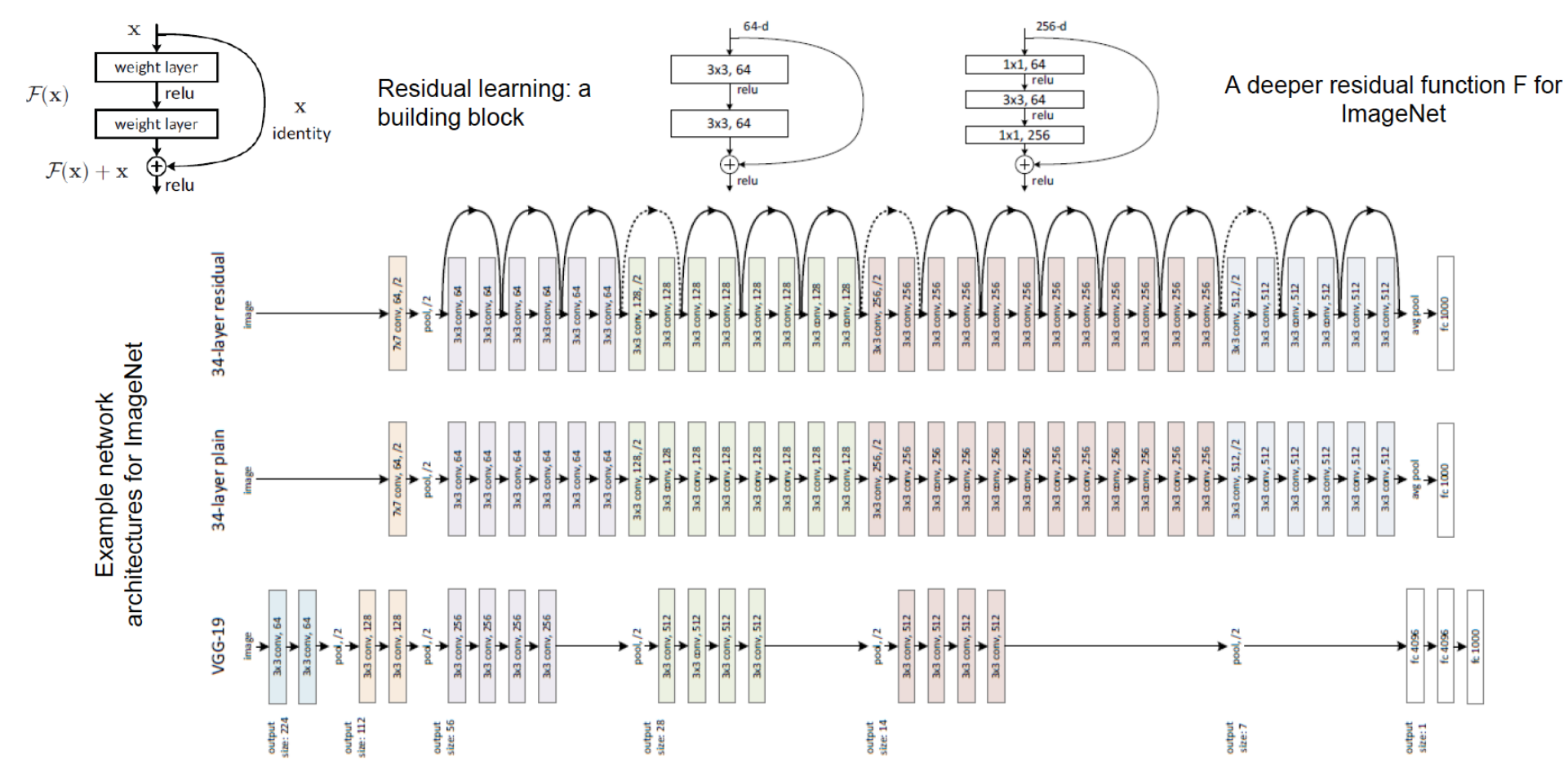
| ConvNet Configuration       |                        |                               |                                            |                                            |                                                         |
|-----------------------------|------------------------|-------------------------------|--------------------------------------------|--------------------------------------------|---------------------------------------------------------|
| A                           | A-LRN                  | B                             | C                                          | D                                          | E                                                       |
| 11 weight layers            | 11 weight layers       | 13 weight layers              | 16 weight layers                           | 16 weight layers                           | 19 weight layers                                        |
| input (224 × 224 RGB image) |                        |                               |                                            |                                            |                                                         |
| conv3-64                    | conv3-64<br>LRN        | conv3-64<br><b>conv3-64</b>   | conv3-64<br>conv3-64                       | conv3-64<br>conv3-64                       | conv3-64<br>conv3-64                                    |
| maxpool                     |                        |                               |                                            |                                            |                                                         |
| conv3-128                   | conv3-128              | conv3-128<br><b>conv3-128</b> | conv3-128<br>conv3-128                     | conv3-128<br>conv3-128                     | conv3-128<br>conv3-128                                  |
| maxpool                     |                        |                               |                                            |                                            |                                                         |
| conv3-256<br>conv3-256      | conv3-256<br>conv3-256 | conv3-256<br>conv3-256        | conv3-256<br>conv3-256<br><b>conv1-256</b> | conv3-256<br>conv3-256<br><b>conv3-256</b> | conv3-256<br>conv3-256<br>conv3-256<br><b>conv3-256</b> |
| maxpool                     |                        |                               |                                            |                                            |                                                         |
| conv3-512<br>conv3-512      | conv3-512<br>conv3-512 | conv3-512<br>conv3-512        | conv3-512<br>conv3-512<br><b>conv1-512</b> | conv3-512<br>conv3-512<br><b>conv3-512</b> | conv3-512<br>conv3-512<br>conv3-512<br><b>conv3-512</b> |
| maxpool                     |                        |                               |                                            |                                            |                                                         |
| conv3-512<br>conv3-512      | conv3-512<br>conv3-512 | conv3-512<br>conv3-512        | conv3-512<br>conv3-512<br><b>conv1-512</b> | conv3-512<br>conv3-512<br><b>conv3-512</b> | conv3-512<br>conv3-512<br>conv3-512<br><b>conv3-512</b> |
| maxpool                     |                        |                               |                                            |                                            |                                                         |
| FC-4096                     |                        |                               |                                            |                                            |                                                         |
| FC-4096                     |                        |                               |                                            |                                            |                                                         |
| FC-1000                     |                        |                               |                                            |                                            |                                                         |
| soft-max                    |                        |                               |                                            |                                            |                                                         |

### Key Technical Features:

- Increase depth (up to 19)
- Smaller filter size (3)

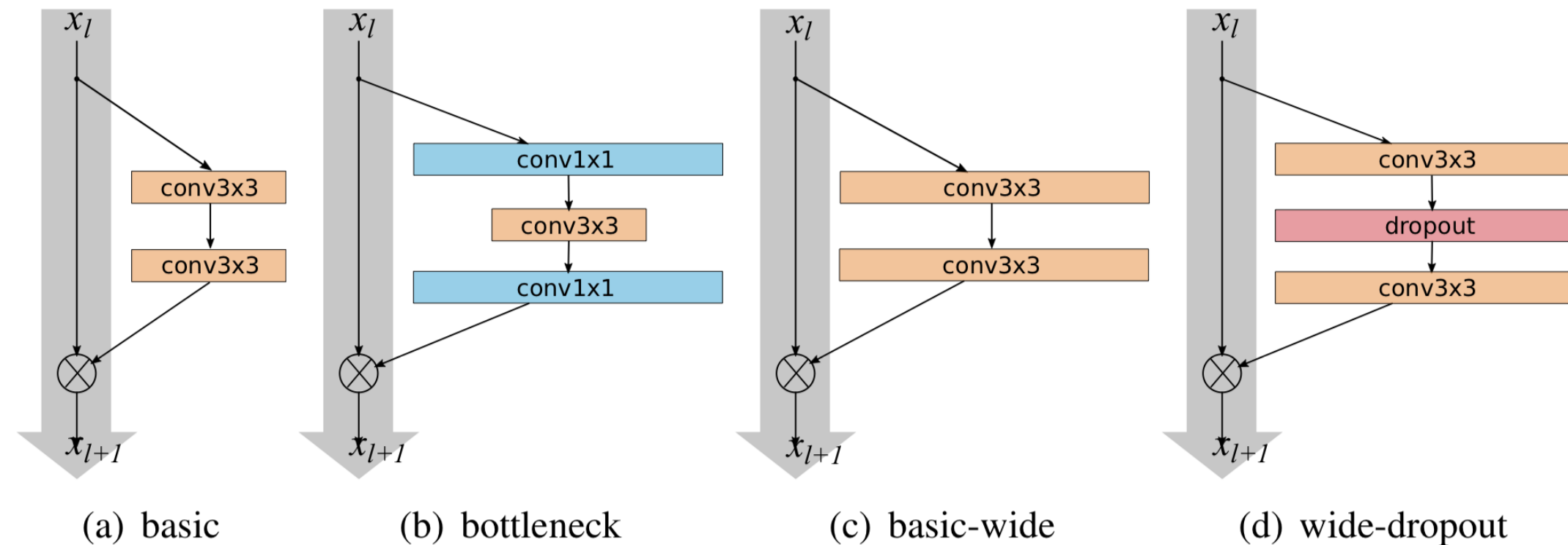
Configurations D and E are widely used for various tasks, called *VGG-16* and *VGG-19*

# Deep Residual Network (ResNet), 2015



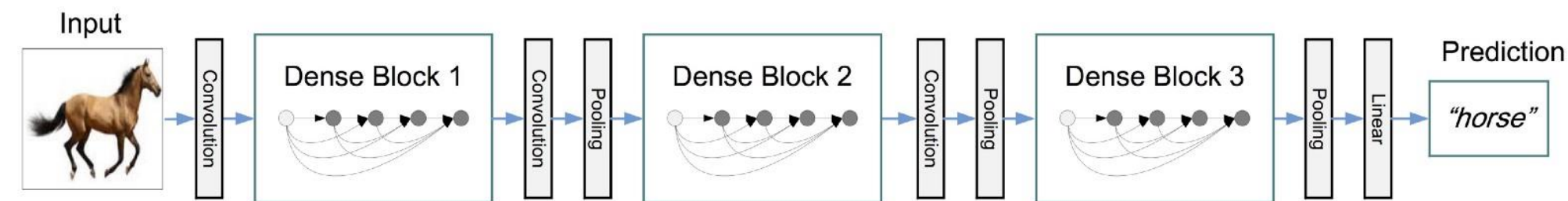
Key Technical Features: skip connections for residual mapping, up to > 1000 layers

# Wide ResNet, 2016



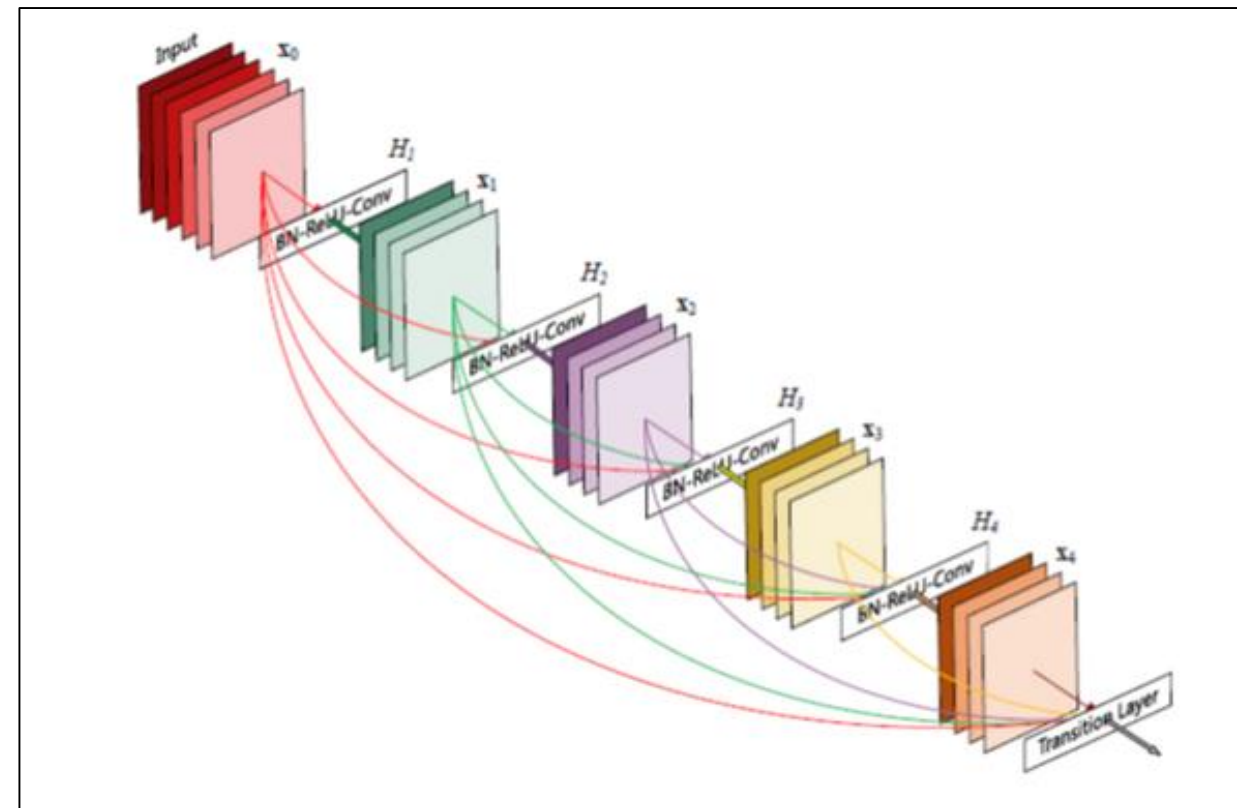
- Widening of ResNet blocks (if done properly) provides a more effective way of improving performance of residual networks compared to increasing their depth.
- A wide 16-layer deep network has the same accuracy as a 1000-layer thin deep network and a comparable number of parameters, although being several times faster to train.

# Densely Connected Convolutional Networks (DenseNet), 2017

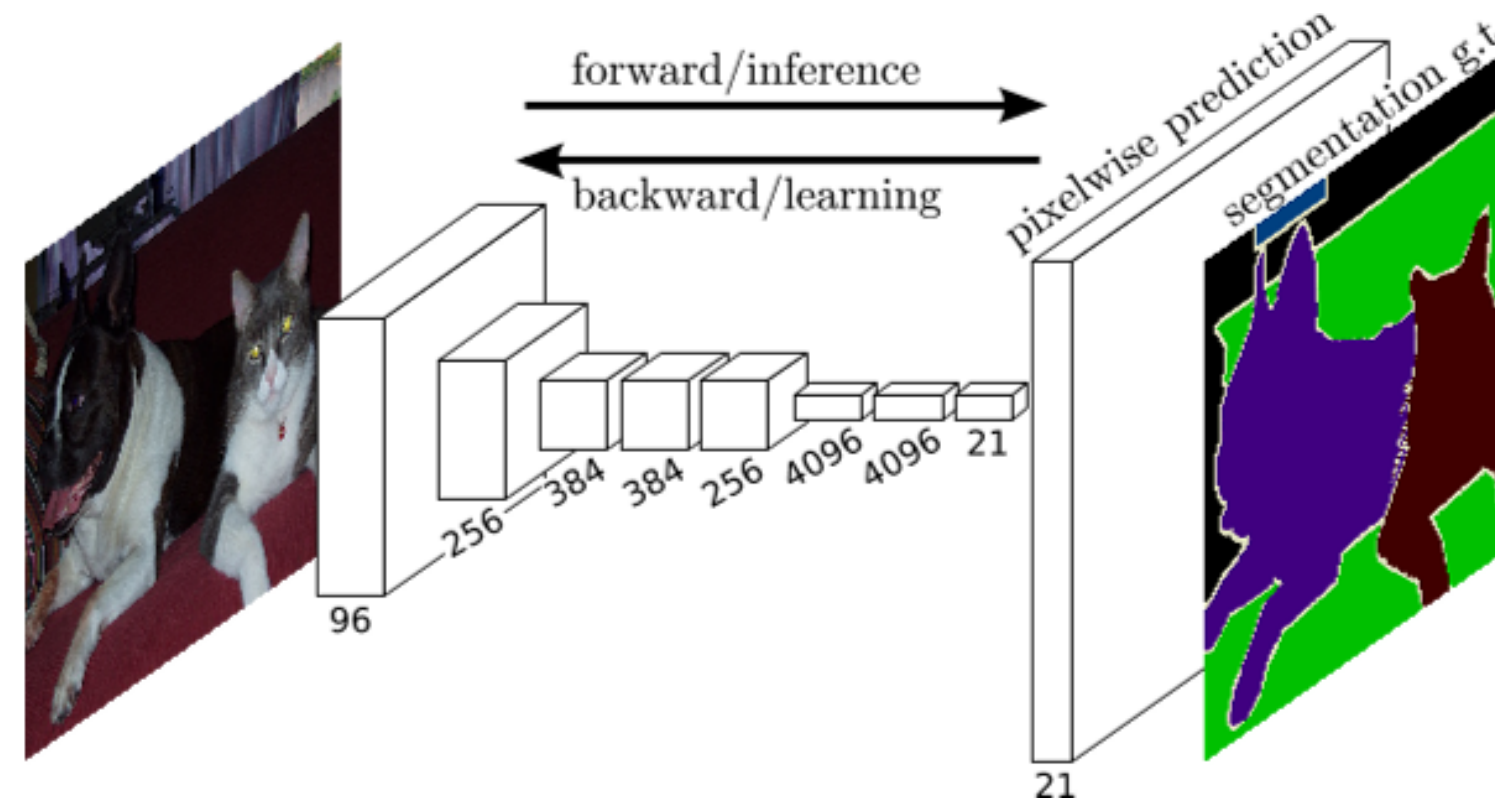


## Key Technical Features:

- Finer combination of multi-scale features (or whatever...)



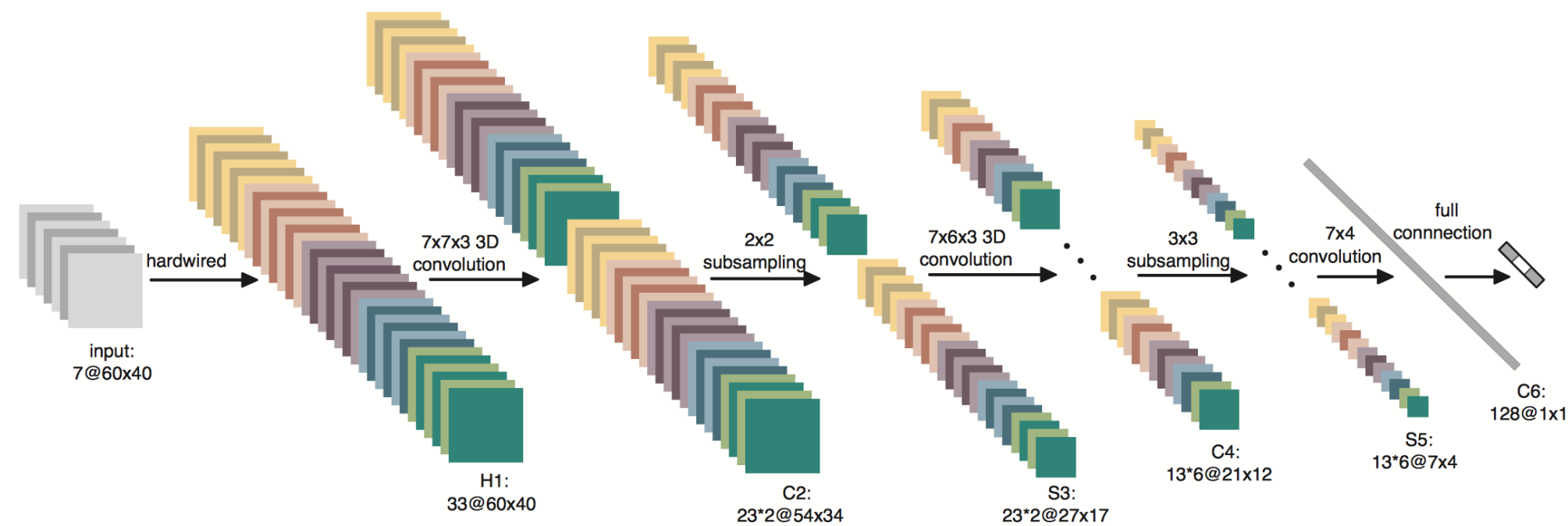
# Fully Convolutional Network (FCN)



## Key Technical Features:

- No fully-connected layer -> No fixed requirement on input size
- Widely adopted in pixel-to-pixel prediction tasks, e.g., image segmentation

# 3D Convolutional Network (3D CNN)



## Key Technical Features:

- Going from 2D convolutional filters to 3D filters, to take temporal coherence into consideration